SITARE University

T0

Version Control Mechanics

Escaping the Final_v2.docx trap — the four places your code lives, and why almost every confusing git error is just two of them disagreeing.

What is version control?

A time machine for your code.

Two ways to keep old work

Save As…

Overwrites the file, or forks a new one with a longer name.

One line of history: whatever you happened to keep.

Undo stops the moment you close the file.

Git

Every save is a complete, separate snapshot.

History branches — old and new versions coexist.

Nothing is discarded until you say so.

Every project has four places code can be

- **Working Directory** — the files on disk, mid-edit.
- **Staging Area** — what you've prepared for the next snapshot.
- **Local Repository** — snapshots git has already saved, on your machine.
- **Remote Repository** — the copy your team agrees is the source of truth.

Almost every confusing git error is a statement that two of these four disagree.

What is the working tree, literally?

Whatever's on disk right now — git's own name for the Working Directory.

A working tree, mid-feature

project/ — working tree

```
project/
├── app/
│   ├── server.py      modified
│   └── routes/
│       └── webhooks.py      untracked
├── tests
│   └── test_server.py      unmodified
└── .env      ignored
```

> Four states, no snapshots. The working tree only ever describes right now.

The same tree, after git add server.py

project/ — working tree

```
project/
├── app/
│   ├── server.py      staged
│   └── routes/
│       └── webhooks.py      untracked
├── tests/
│   └── test_server.py      unmodified
└── .env      ignored
```

> Only server.py changed state. webhooks.py is still untracked — staging one file never touches another.

T0 · L1 / PREDICT BEFORE YOU RUN IT

git status says nothing to commit — but a teammate can't see your latest change on GitHub. Which two locations disagree?

A Working Directory vs. Staging Area

B Local Repository vs. Remote Repository

C Staging Area vs. Local Repository

HANDS UP. COMMIT BEFORE THE REVEAL.

T0 · L1 / PREDICT BEFORE YOU RUN IT

git status says nothing to commit — but a teammate can't see your latest change on GitHub. Which two locations disagree?

A Working Directory vs. Staging Area

B Local Repository vs. Remote Repository

C Staging Area vs. Local Repository

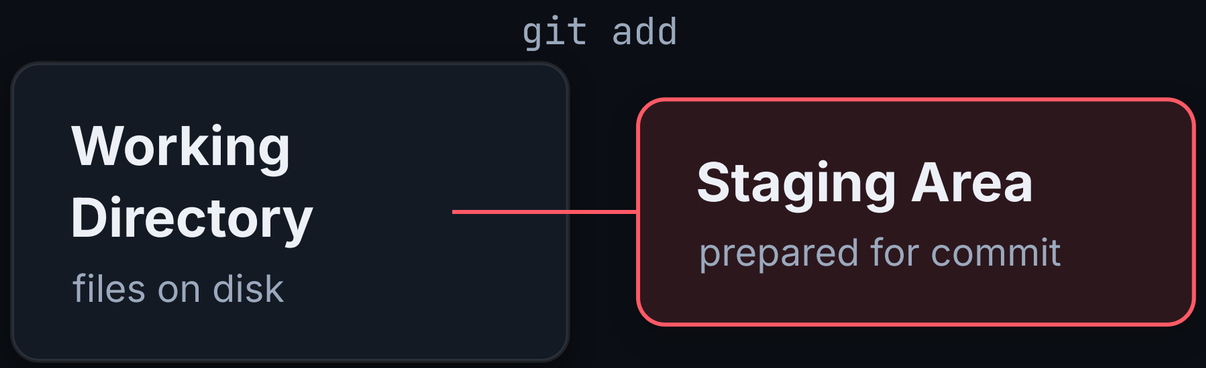
GIT STATUS COMPARES THE FIRST THREE LOCATIONS — NEVER THE REMOTE. A CLEAN STATUS ONLY MEANS THOSE THREE AGREE. WHETHER THE REMOTE HAS CAUGHT UP IS A SEPARATE QUESTION: GIT PUSH.

Start at the Working Directory

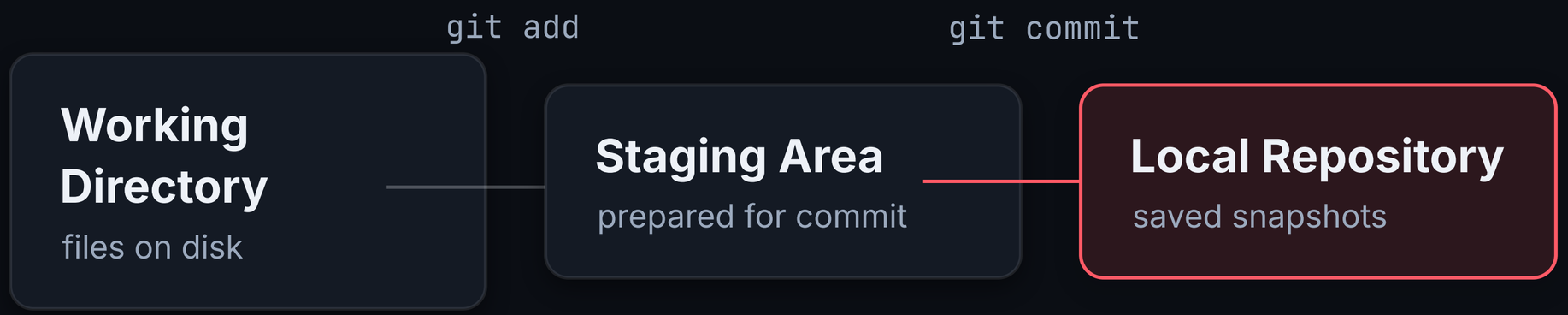
Working Directory

files on disk

git add stages your changes

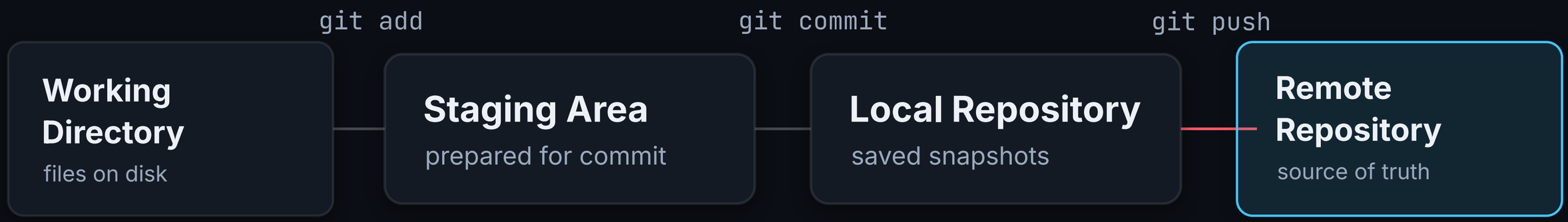


git commit saves to the Local Repository

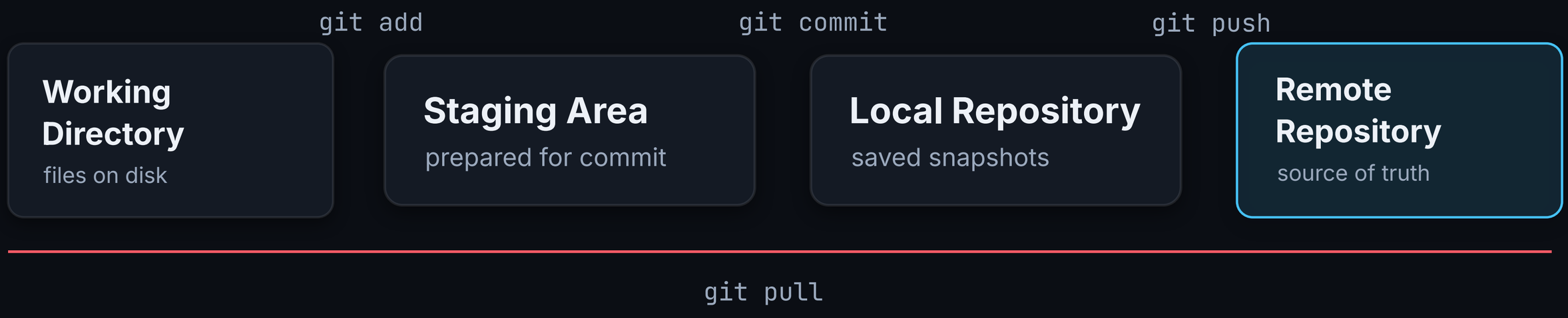


T0 · L1 / THE FOUR LOCATIONS

git push crosses the network to Remote



git pull brings Remote's snapshots back



Files here are untracked, or modified — never "saved"

- **Untracked** — git has never seen this file. It doesn't exist to version control yet.
- **Modified** — git has seen it before, and the copy on disk no longer matches the last snapshot.
- Neither state means the file is backed up anywhere but your disk.

> The Working Directory is just your filesystem. Git is only watching.

T0 · L1 / WORKING DIRECTORY

git status reports what changed

you — project — 80×24

```
$  git status
On branch main
Changes not staged for commit:
  modified:   app/server.py
Untracked files:
  routes/webhooks.py
```

What does git init actually build?

A hidden database.

One command creates the whole apparatus

you — project — 80×24

```
$ git init
Initialized empty Git repository in /project/.git/

$ ls -la .git
HEAD
config
description
hooks/
objects/
refs/
```

T0 · L1 / STAGING AREA

A photography studio, not a filing cabinet

Working Directory

The chaotic backstage — everything you're touching, in whatever state it's in.

Staging Area

The lit stage — only what you've deliberately arranged, ready for the photo.

T0 · L1 / STAGING AREA

git add freezes a file's current state

you — project — 80×24

```
$  git status
Changes not staged for commit:
  modified:   app/server.py

$  git add app/server.py

$  git status
Changes to be committed:
  modified:   app/server.py
```

THE CS UNDERNEATH

T0 · L1 / LOCAL REPOSITORY

A commit is an immutable snapshot

```
commit = snapshot + parent hash
```

- Every commit points to **exactly one parent** — the commit before it.
- Follow the parent pointers back and you get the entire history, in order.
- **Immutable** doesn't mean permanent — it means history is only ever appended to, never quietly edited.

Change anything about a commit and its hash changes — which is why "editing history" always means replacing commits, not altering them.

T0 · L1 / LOCAL REPOSITORY

git commit writes the snapshot permanently

you — project — 80×24

```
$  git commit -m "fix(server): handle empty payload"
[main 8b41d07] fix(server): handle empty payload
 1 file changed, 4 insertions(+), 1 deletion(-)
```

T0 · L1 / LOCAL REPOSITORY

Every commit points to the one before it



What makes a commit message useful?

Not what changed. Why.

T0 · L1 / COMMIT HYGIENE

One commit, one idea

Bad

One commit touching auth, a styling tweak, and a typo fix at once.

Subject line: "updates" — describes nothing.

No body. Reviewers guess at intent months later.

Good

One commit, one logical change — the auth fix, nothing else.

Subject line, imperative: "Fix session timeout on token refresh."

Body explains **why** the bug happened — not a restatement of the diff.

T0 · L1 / COMMIT HYGIENE

The subject says what. The body says why.

you — project — 80×24

```
$  git log -1

commit 8b41d07
fix(auth): Fix session timeout on token
    Refresh tokens were checked against the old
    expiry before the new one was saved — every
    refresh looked expired one request late.
```

T0 · L1 / PREDICT BEFORE YOU RUN IT

You edit three files, but git add only one of them. What does git commit snapshot?

A Only the one staged file

B All three edited files

C Nothing, since two are still unstaged

HANDS UP. COMMIT BEFORE THE REVEAL.

T0 · L1 / PREDICT BEFORE YOU RUN IT

You edit three files, but git add only one of them. What does git commit snapshot?

A Only the one staged file

B All three edited files

C Nothing, since two are still unstaged

A COMMIT CAPTURES EXACTLY WHAT'S IN THE STAGING AREA — NOT EVERYTHING YOU'VE TOUCHED. THE OTHER TWO EDITS WAIT IN THE WORKING DIRECTORY FOR THEIR OWN GIT ADD.

A remote is just another repository, on a server

- GitHub and GitLab don't run different software — they host the same repository format you have locally.
- **"origin"** is a name, not a location. It's whichever remote you set up first.
- The remote only knows what's been pushed to it — it has no idea what's sitting on your laptop.

> Your team agrees the remote is the truth. Git doesn't enforce that — you do.

git push updates the remote's pointers to match yours



Two commands, one letter of difference, very different effects

git fetch

Downloads the remote's new commits and metadata.

Your Working Directory is untouched.

git pull

Does a fetch, then immediately merges into your current branch.

Your Working Directory changes.

fetch looks. pull looks and merges.

you — project — 80×24

```
$  git fetch origin
remote: Enumerating objects: 5, done.
 main → origin/main   (working directory untouched)

$  git pull origin main
Updating 3f9a1c2..8b41d07
Fast-forward — app/server.py updated on disk
```

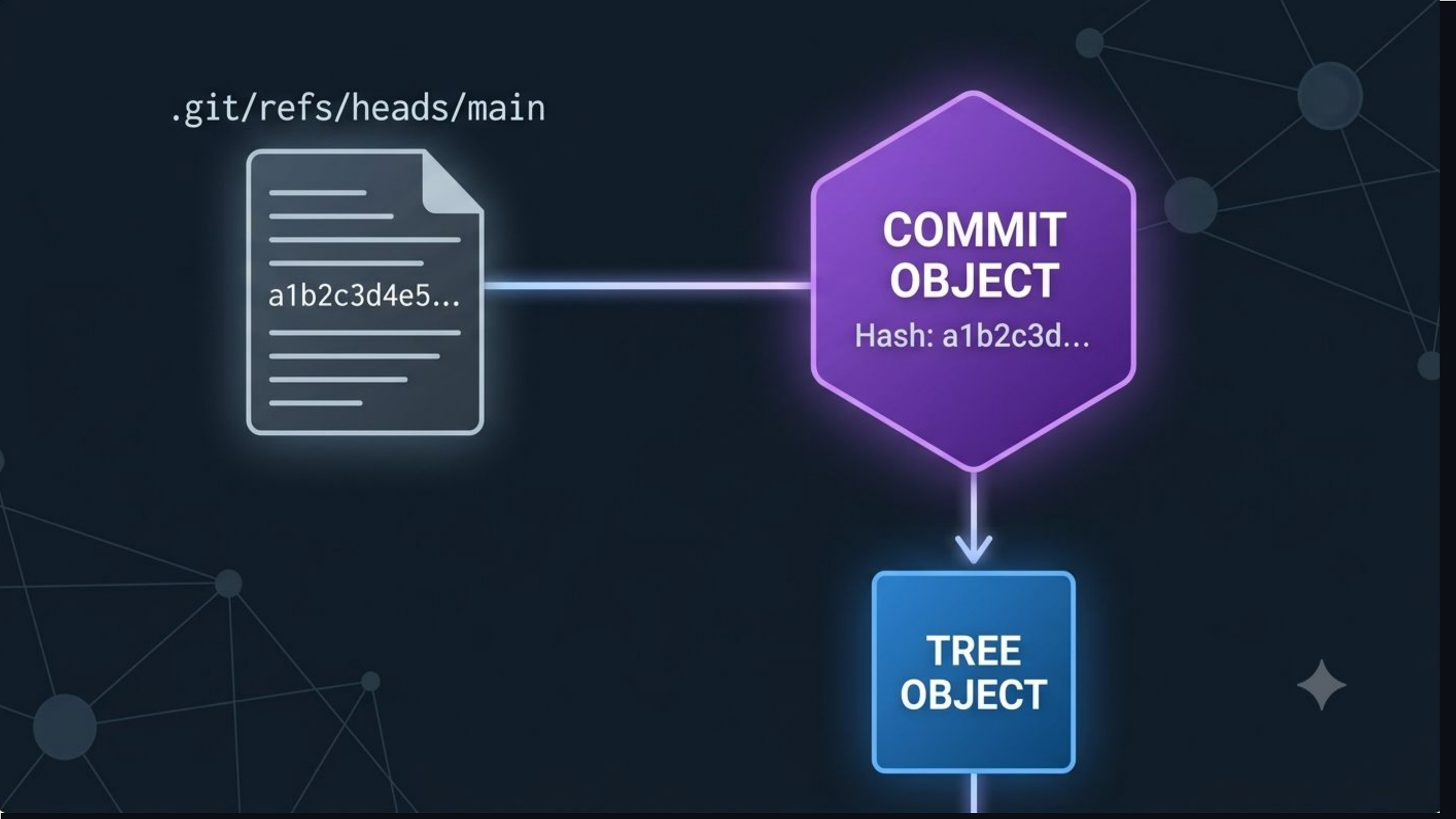
What is a Branch?

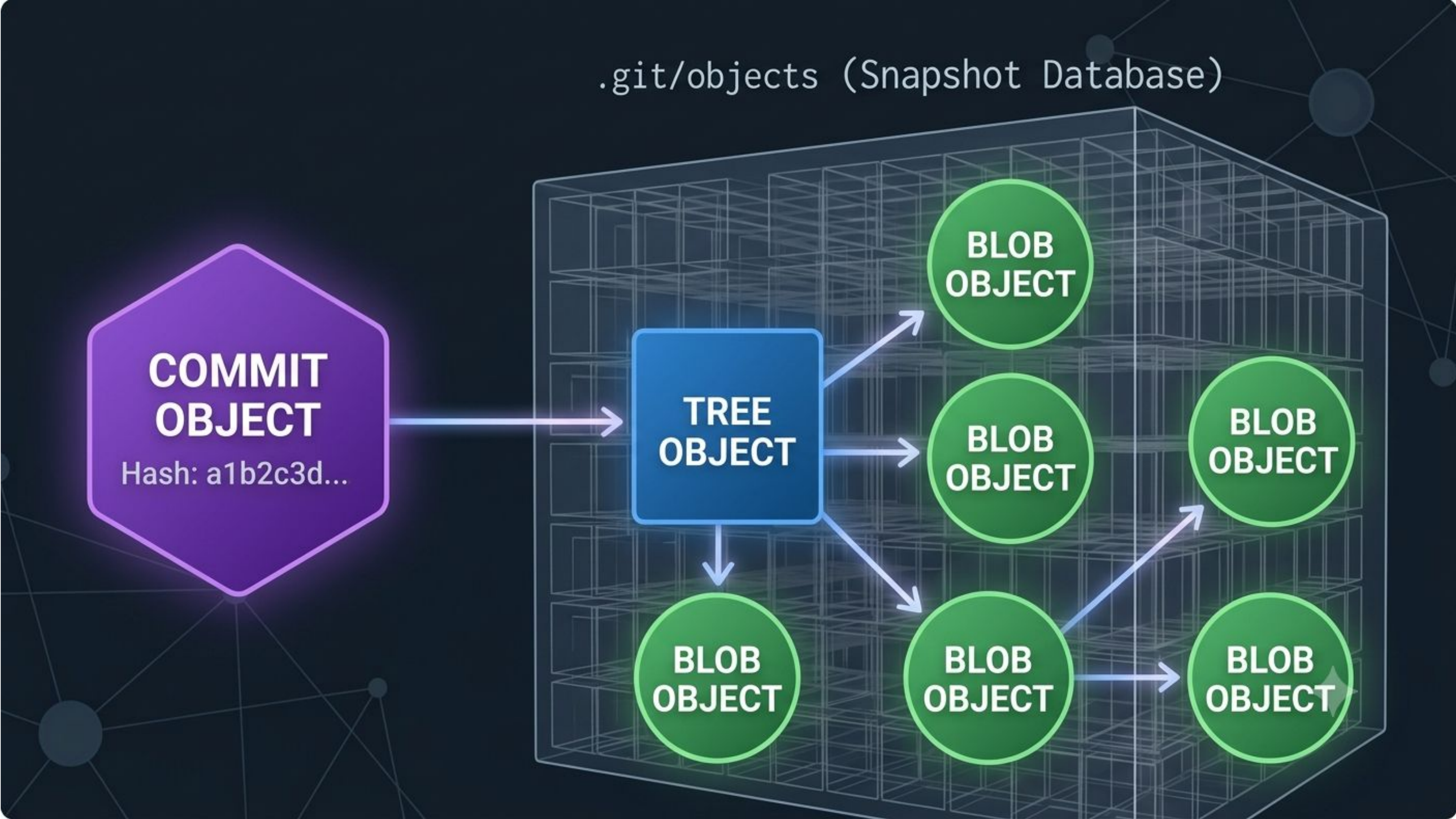
Just a 40-character text file.

A branch is a movable pointer, not a copy

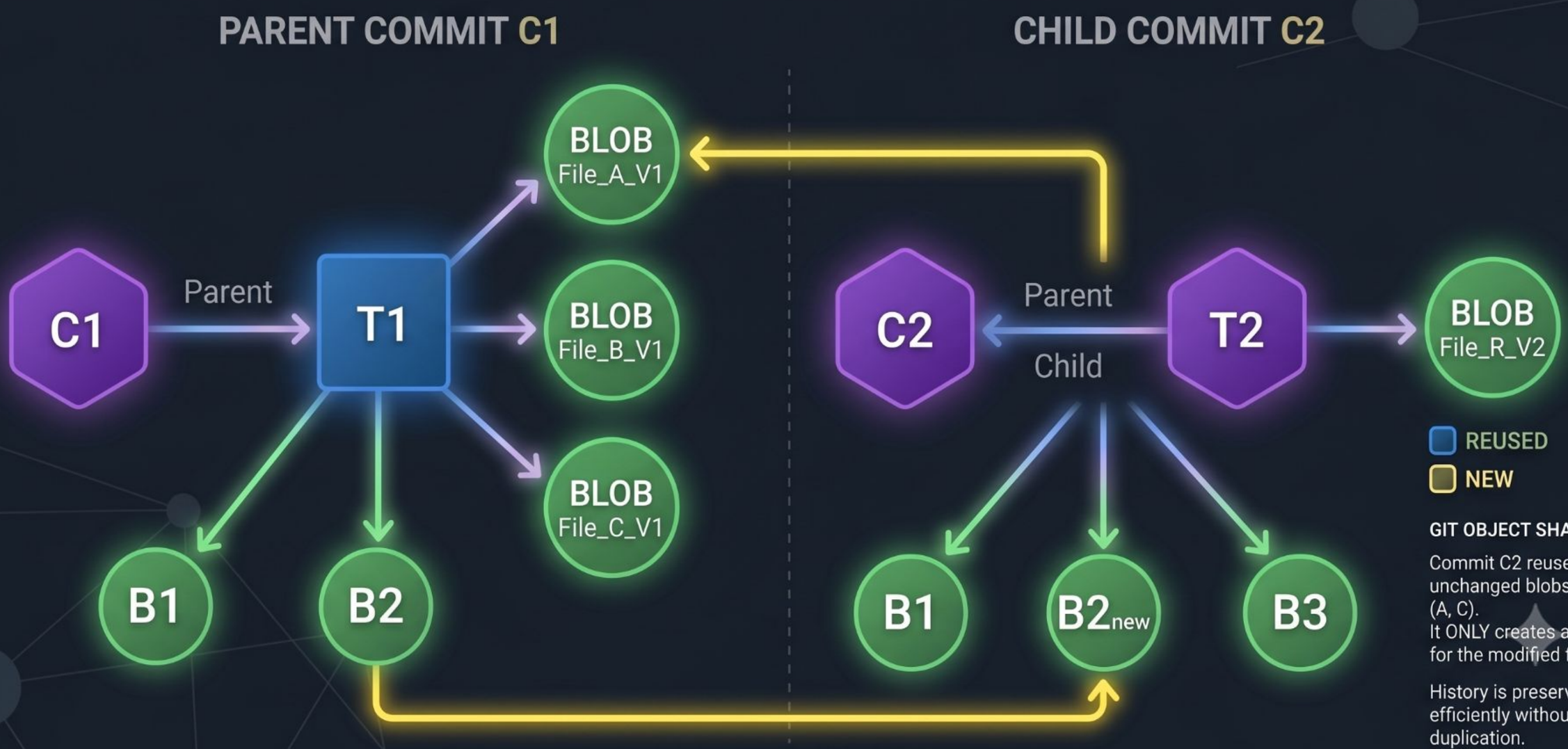
- A branch is a 40-character text file holding **one commit hash**. Nothing else.
- Committing on a branch just rewrites that one file to point at the new commit.
- No files are duplicated. The branch is a label on the graph, not a second copy of it.

Creating a branch means writing one small file — which is why it's instantaneous.


.git/refs/heads/main
a1b2c3d4e5...
COMMIT OBJECT
Hash: a1b2c3d...
TREE OBJECT


.git/objects (Snapshot Database)
COMMIT OBJECT
Hash: a1b2c3d...
TREE OBJECT
BLOB OBJECT
BLOB OBJECT
BLOB OBJECT
BLOB OBJECT
BLOB OBJECT
BLOB OBJECT

UNDERSTANDING OBJECT SHARING & DATA DEDUPLICATION:
HOW GIT STORES DIFFERENCES EFFICIENTLY.



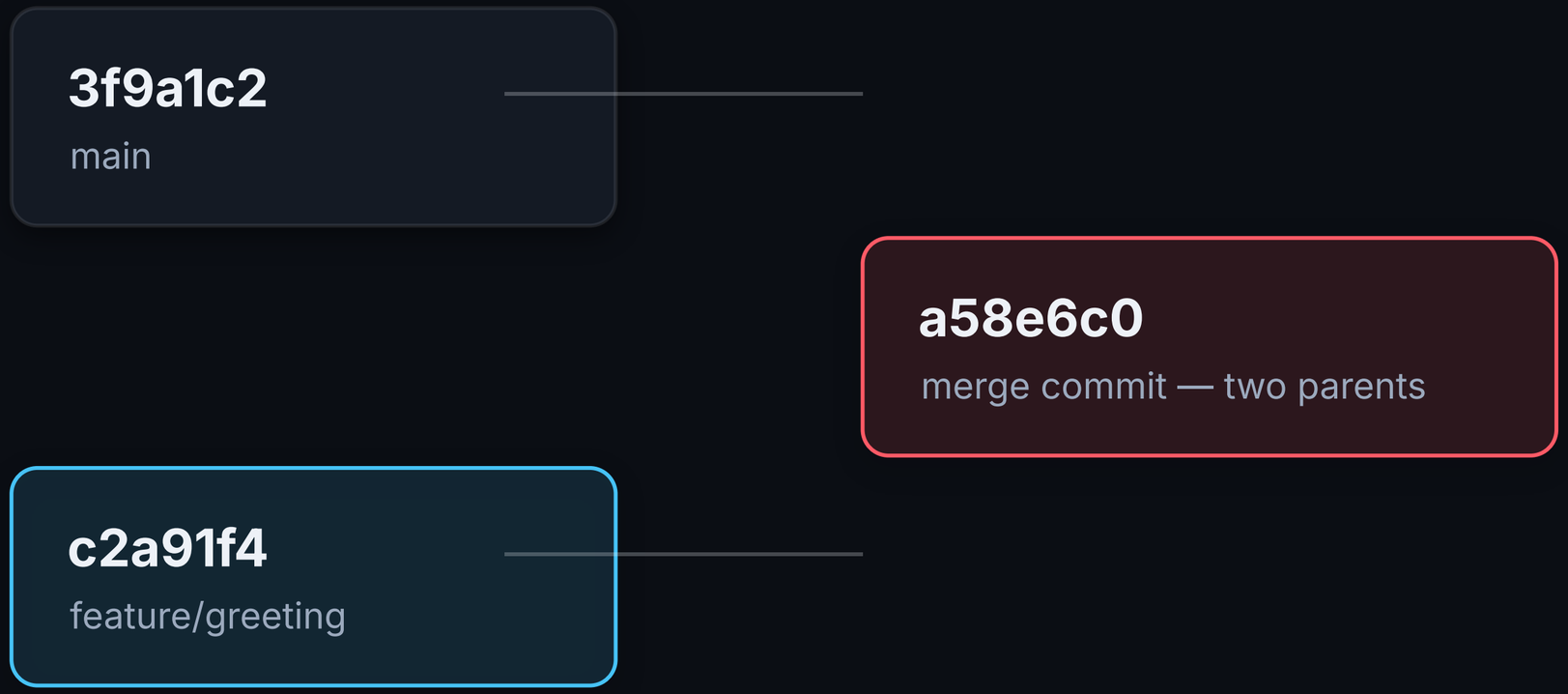
Why a branch costs nothing to create

- Creating a branch writes a pointer file. Deleting one removes it. Both are instant.
- Older centralized systems like SVN copied the entire directory tree per branch.
- That cost was high enough that teams avoided branching — and worked around each other on one line instead.

> Git made branching free, so teams stopped avoiding it.

T0 · L1 / MERGE

Merging joins two histories into one



Why does git refuse to guess?

Two branches changed the same lines.

T0 · L1 / MERGE CONFLICTS

Every merge starts as three versions of one file

base
common ancestor

ours
HEAD

theirs
incoming branch

diffed region by region

- one side changed it → take that side
- both sides changed it → conflict

The refusal is structural, not a failure

```
conflict = same region + both sides changed it
```

- Git diffs **ours** against **base**, and **theirs** against **base**, region by region.
- If only one side touched a region, git takes that side automatically — this is why most merges produce no conflict at all.
- If both sides touched the **same** region, git now holds two valid edits and has no rule for choosing between them.

Two edits, one region, no tiebreaker — that's the whole refusal. How git decides what counts as "the same region" is the diff algorithm itself, deferred to T2.

Reading a conflict, line by line

```
<<<<<<< HEAD
return f"Hello, {name}!"
=======
return f"Hi there, {name}."
>>>>>>> feature/greeting
```

> Everything between here and the ======= divider is your branch's version — the one checked out as HEAD.

Reading a conflict, line by line

```
1 <<<<<<< HEAD
2 return f"Hello, {name}!"
3 =======
4 return f"Hi there, {name}."
5 >>>>>>> feature/greeting
```

> The divider itself. No code — it just marks where your version ends and theirs begins.

Reading a conflict, line by line

```
1 <<<<<<< HEAD
2 return f"Hello, {name}!"
3 =======
4 return f"Hi there, {name}."
5 >>>>>>> feature/greeting
```

> Everything up to the divider from here is the incoming branch's version — feature/greeting. Git shows both and steps back.

T0 · L1 / PREDICT BEFORE YOU RUN IT

You stage a file, then keep editing it before committing. git status now lists it as both staged and modified. What fixes the mismatch?

A git commit -a

B git add the file again

C git init

HANDS UP. COMMIT BEFORE THE REVEAL.

SEEN BEFORE · T0 · L0

T0 · L1 / GITIGNORE

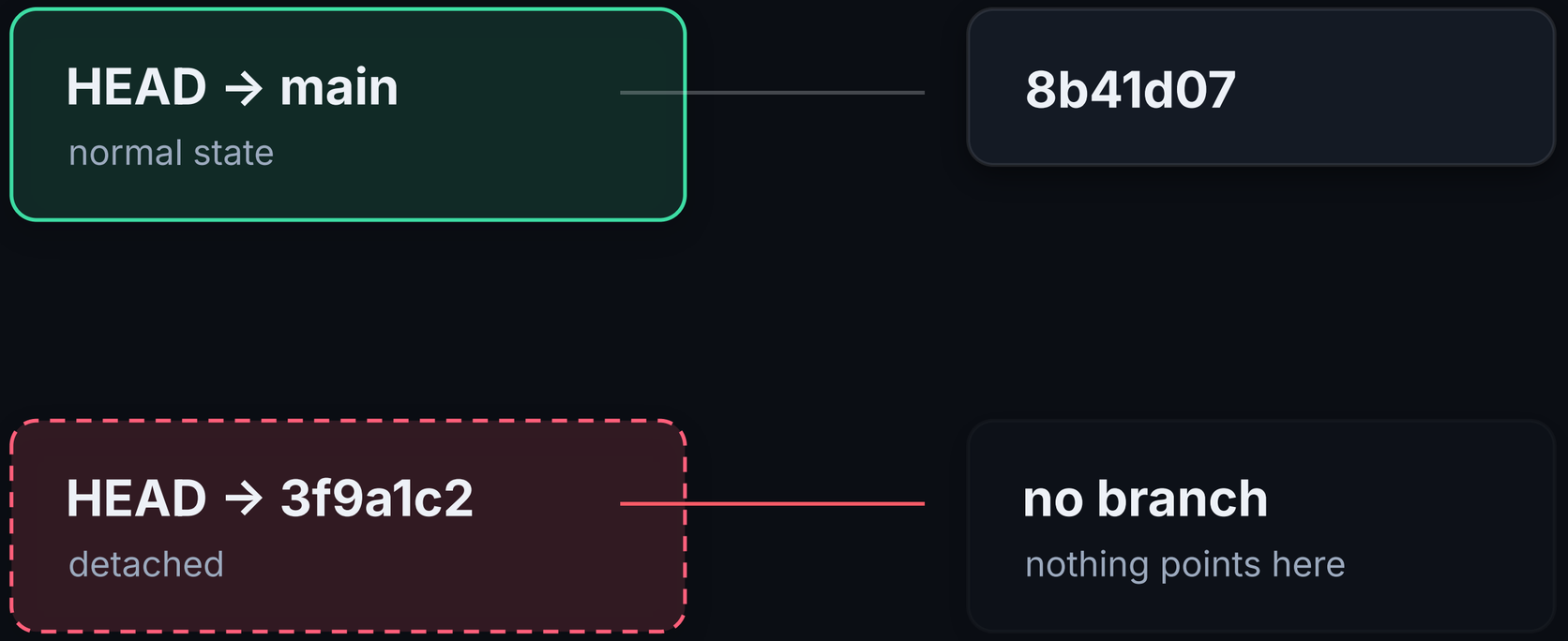
Some files should never reach the Staging Area

- Dependency folders like **node_modules** — regenerable, and enormous.
- Build artifacts and local logs — different on every machine.
- Environment files and credentials — **the same rule from T0 · L0** : never committed, never pushed.

> .gitignore doesn't delete anything. It just tells git add to skip these paths.

T0 · L1 / DETACHED HEAD

HEAD can point at a branch, or skip it



A desk drawer for half-finished work

you — project — 80×24

```
$  git stash
Saved working directory and index state WIP on main

$  git status
nothing to commit, working tree clean

$  git stash pop
  modified:   app/server.py
```

T0 · L1 / RECAP

A teammate's git push is rejected: "failed to push some refs." Local and Remote have diverged. What do they run first?

A git push --force

B git pull, then push again

C git init

HANDS UP. COMMIT BEFORE THE REVEAL.

T0 · L1 / RECAP

A teammate's git push is rejected: "failed to push some refs." Local and Remote have diverged. What do they run first?

A git push --force

B git pull, then push again

C git init

THE REJECTION MEANS THE REMOTE HAS COMMITS THE LOCAL REPOSITORY DOESN'T. FORCE-PUSHING OVERWRITES AND CAN ERASE A TEAMMATE'S WORK — PULLING FIRST GIVES THE PUSH A COMMON ANCESTOR TO BUILD ON.

T0 · L1 / DELIVERABLE

Upcoming week's practical

01 Push a local branch to the remote.

02 Open a pull request against main.

03 Deliberately edit a line a teammate touched - on purpose, to create a merge conflict.

04 Resolve it, and write down which locations disagreed at each step.

This is graded.

What today added to the four locations

- A file moves from Working Directory to Staging with git add — and Staging is a copy, not a promise.
- A commit is an immutable snapshot with a parent — history only grows, it doesn't rewrite itself quietly.
- A branch is a pointer, not a copy — which is why creating one is instant.
- A merge conflict isn't git failing. It's git correctly refusing to guess.

QUESTIONS?